

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ
FACULTAD DE CIENCIAS E INGENIERÍA

ALGORITMIA Y ESTRUCTURA DE DATOS
Segundo Examen
(Segundo Semestre 2025)

Duración: 2h 50 min.

- **No puede utilizar apuntes, solo hojas sueltas en blanco.**
- En cada función el alumno deberá incluir, a modo de comentario, la forma de solución que utiliza para resolver el problema. De no incluirse dicho comentario, el alumno perderá el derecho a reclamo en esa pregunta.
- No puede emplear plantillas o funciones no vistas en los cursos de programación de la especialidad.
- Los programas deben ser desarrollados en el lenguaje C++. Si la implementación es diferente a la estrategia indicada o no la incluye, la pregunta no será corregida.
- Un programa que no muestre resultados coherentes y/o útiles será corregido sobre el 50% del puntaje asignado a dicha pregunta.
- Debe utilizar comentarios para explicar la lógica seguida en el programa elaborado. El orden será parte de la evaluación.
- **Solo está permitido acceder a la plataforma de PAIDEIA, cualquier tipo de navegación, búsqueda o uso de herramientas de comunicación se considera plagio por tal motivo se anulará la evaluación y se procederá con las medidas disciplinarias dispuestas por la FCI.**
- Para esta evaluación solo se permite el uso de las librerías `iostream`, `iomanip`, `limits`, `cstring`, `cmath` o `fstream`
- Su trabajo deberá ser subido a PAIDEIA.
- **Es obligatorio usar como compilador CLion.**
- Los archivos deben llevar como nombre su código de la siguiente forma `codigo_EX2_P#` (donde # representa el número de la pregunta)

Elija las preguntas para obtener un puntaje de 20, si escoge más de esa nota se considera la respuesta con más bajo puntaje:

Pregunta 1 (5 puntos)

Un sistema de diagnóstico odontológico analiza radiografías y genera una secuencia de valores numéricos que representan la **densidad mineral del esmalte** a lo largo de un diente, desde la **superficie externa** hacia la **pulpa dental**.

- Valores **altos** $\Rightarrow$ esmalte sano
- Valores **bajos** $\Rightarrow$ posible caries (desmineralización)

Gracias al procesamiento de imagen, el sistema devuelve un arreglo de n valores **ordenado de forma decreciente** (es decir, cada valor es menor o igual al anterior):

Al inicio (posición 0) suelen estar las zonas más sanas y, conforme avanzamos en el arreglo, pueden aparecer zonas con caries.

La clínica define un **umbral crítico de caries con un valor de T** :

- Si la densidad baja de T , se considera que hay **caries clínica**.

La jefa de diagnóstico necesita identificar la **primera posición** del diente donde la densidad cae por debajo del umbral, para decidir el **punto exacto** a intervenir.

Por ejemplo:

Se tiene un arreglo de 8 densidades:

Entrada:

$n=8$

arreglo = 98, 95, 93, 90, 72, 60, 55, 50

$T = 80$

La salida sería:

Primera zona con caries clínica detectada en la posición 4 (Recuerde que la posición inicia en 0)

Densidad Registrada: 72

Otro ejemplo:

Entrada:

$n=8$

arreglo = 98, 95, 93, 90, 72, 60, 55, 50

$T = 40$

La salida sería:

No se detecta caries clínica.

Se le pide implementar una función recursiva, aplicando la técnica de divide y vencerás que encuentre a primera posición donde $D[i] < T$, con una complejidad $O(\log n)$. Si no existe ninguna posición con $D[i] < T$, se debe indicar que no se detecta caries clínica. **Para la resolución de este problema no puede utilizar una proposición lógica compuesta como parte de una condición, por lo que solo puede utilizar una proposición lógica simple, del tipo $D[i] < T$ u otra proposición lógica simple del tipo $D[i] > T$.**

Pregunta 2 (5 puntos)

Un equipo de ciberseguridad ha detectado que el log de actividades de usuarios del sistema crítico de la empresa fue manipulado mediante una rotación: el primer registro real del día ya no aparece al inicio del arreglo. Cada registro del log contiene dos valores: (i) **idActividad**: entero positivo, todos distintos; originalmente en orden estrictamente creciente, y (ii) **riesgo**: entero (positivo, cero o negativo) que representa el nivel de riesgo generado por la actividad del usuario en ese minuto.

Se tiene un solo arreglo bidimensional definido directamente en el main(), donde:

$A[][] =$	10	20	30	40	50	60	70	80
	-4	5	1	-2	4	-1	3	-2

Sin embargo, debido al ataque, este arreglo es una versión rotada del original.

$$A[][] = \begin{array}{|c|c|c|c|c|c|c|c|} \hline 50 & 60 & 70 & 80 & 10 & 20 & 30 & 40 \\ \hline 4 & -1 & 3 & -2 & -4 & 5 & 1 & -2 \\ \hline \end{array}$$

Se le pide implementar un programa en C++, utilizando la estrategia Divide y Vencerás para que ayude al equipo de ciberseguridad a calcular: la máxima acumulación de riesgo en un intervalo continuo pero una vez corregido el orden cronológico (es decir, después de “desrotar” el arreglo). La complejidad no debe superar $O(n \log n)$.

Caso de prueba 1: $\{\{50, 4\}, \{60, -1\}, \{70, 3\}, \{80, -2\}, \{10, -4\}, \{20, 5\}, \{30, 1\}, \{40, -2\}\}$;

	0	1	2	3	4	5	6	7
$A[][] =$	50	60	70	80	10	20	30	40
	4	-1	3	-2	-4	5	1	-2

$A[][] =$	10	20	30	40	50	60	70	80
	-4	5	1	-2	4	-1	3	-2

Salidas:

Posición inicial original: 4

Máxima acumulación de riesgo en un intervalo continuo: 10

Caso de prueba 2:

$\{\{80, -4\}, \{90, -2\}, \{10, 1\}, \{20, -3\}, \{30, 6\}, \{40, -8\}, \{50, 5\}, \{60, 2\}, \{70, -1\}\}$;

	0	1	2	3	4	5	6	7	8
$A[][] =$	80	90	10	20	30	40	50	60	70
	-4	-2	1	-3	6	-8	5	2	-1

$A[][] =$	10	20	30	40	50	60	70	80	90
	1	-3	6	-8	5	2	-1	-4	-2

Salidas:

Posición inicial original: 2

Máxima acumulación de riesgo en un intervalo continuo: 7

Caso de prueba 3:

$\{\{40, 6\}, \{50, -4\}, \{60, -9\}, \{70, 0\}, \{80, 7\}, \{90, -1\}, \{100, 4\}, \{10, -3\}, \{20, 1\}, \{30, 4\}\}$;

	0	1	2	3	4	5	6	7	8	9
A[0][0] =	40	50	60	70	80	90	100	10	20	30
	6	-4	-9	0	7	-1	4	-3	1	4

	10	20	30	40	50	60	70	80	90	100
A[1][0] =	10	20	30	40	50	60	70	80	90	100
	-3	1	4	6	-4	-9	0	7	-1	4

Salidas:

Posición inicial original: 7

Máxima acumulación de riesgo en un intervalo continuo: 11

Pregunta 3 (5 puntos)

Un **Min-Heap** (o montículo mínimo) es un árbol binario que cumple dos reglas:

1. **Forma de árbol casi completo:** Todos los niveles están completos, excepto posiblemente el último, que se llena de izquierda a derecha.
2. **Propiedad de montículo mínimo:** El valor de cada nodo es menor o igual que el de sus hijos. Por lo tanto, el valor más pequeño siempre está en la raíz.

Estructura de nodo: Cada nodo contiene:

- Valor almacenado
- Referencias a hijo izquierdo y derecho
- Referencia al nodo **padre**

Dado un **Min-Heap ya construido de nivel 3**, implementa la función **eliminarMinimo**.

La función eliminar debe realizar lo siguiente:

- Eliminar y devolver el elemento mínimo (que está en la raíz).
- Para eliminarlo, reemplazar la raíz con **la hoja más a la derecha del último nivel**.
- Luego, **reajustar** el heap para que vuelva a cumplir la propiedad de montículo mínimo, comparando el nuevo valor de la raíz con sus hijos e intercambiándolo con el menor de ellos, repitiendo este proceso hacia abajo hasta que el elemento esté en su posición correcta.

Tips (Recomendado):

1. Usar BFS con cola para encontrar la última hoja, ó puede usar otro TAD.
2. Aprovechar la referencia al padre para:
 - Desconectar fácilmente la última hoja
 - Facilitar el proceso de hundimiento (reajuste)

Heap inicial

```

      2 ← Elemento que hay que eliminar
     / \
    5   3
  
```

```

  / \   / \
 9  8 7  6 ← Sustituto

```

Antes de la reordenación

```

    6
   / \
  5   3
 / \   /
9  8 7

```

Después de la reordenación

```

    3
   / \
  5   6
 / \   /
9  8 7

```

Pregunta 4 (10 puntos)

Estadísticas de orden en un Árbol Binario de Búsqueda Balanceado

Se le entrega un programa base en C++ que implementa un **Árbol ABB balanceado**. El proyecto incluye funciones básicas para el manejo del ABB. Sin embargo, falta completar la **inserción** y la **eliminación**.

Se le pide las siguientes tareas:

1. Completar funciones Inserción y Eliminación: (1 pto)

Complete y pruebe las funciones insertar y **eliminarNodo**.

2. int k_esimo_menor(Nodo* raiz, int k); (2ptos)

La estructura de cada nodo del AVL incluye el campo:

int size; // cantidad de nodos en el subárbol

Usando este campo, implemente un algoritmo que retorne el **k-ésimo elemento menor** del árbol, cumpliendo:

- Debe ejecutarse en **$O(\log n)$** .
- Debe validar que k se encuentra entre 1 y size(raiz).

Lógica esperada:

- Sea $L = \text{size}(\text{raiz} \rightarrow \text{izq})$
 - Si $k == L + 1 \rightarrow$ el nodo actual es el k-ésimo menor.
 - Si $k \leq L \rightarrow$ buscar en el subárbol izquierdo.
 - Si $k > L + 1 \rightarrow$ buscar en el subárbol derecho usando $k - (L + 1)$.

Elementos en orden (in-order):	En este ejemplo:
3 5 8 10 13 15 20	El 1° menor es 3
	El 2° menor es 5
	El 3° menor es 8
1° 2° 3° 4° 5° 6° 7°	El 4° menor es 10
	... y así sucesivamente.

3. double mediana(Nodo* raiz); (2ptos)

La mediana del árbol se define como:

- Si n (cantidad total de nodos) es impar $\rightarrow$
La mediana es el elemento ubicado en la posición $(n + 1) / 2$.
- Si n es par $\rightarrow$
La mediana es el **promedio** de los elementos en posiciones $n/2$ y $n/2 + 1$.

Use su función `k_esimo_menor` para implementar esta operación.

La función debe lanzar una excepción si el árbol está vacío.

Pregunta 5 (5 puntos)

Se desea invertir los primeros k números de una cola. Por tal motivo debe desarrollar un algoritmo que realice esta operación sin usar ninguna estructura auxiliar. La complejidad de este algoritmo debe ser $O(n)$.

A continuación, un ejemplo:

Si se tiene la siguiente cola: [1, 2, 3, 4, 5]

Para un $k = 3$

El resultado debe ser: [3, 2, 1, 4, 5]

Al finalizar el examen, comprima la carpeta de su proyecto empleando el programa Zip que viene por defecto en el Windows, **no se aceptarán los trabajos compactados con otros programas como RAR, WinRAR, 7zip o similares**. Luego súbalo a la tarea programa en Paideia para este examen.

Profesores del curso: Ana Roncal
Fernando Huamán
David Allasi
Erasmó Gómez
Rony Cueva

San Miguel, 05 de diciembre del 2025